Universe of Goo

Milestone I

Due: Februrary 19, 2026

1 Overview of Milestone

In this project, you will be implementing the basic physics behind the indie game *World of Goo*. The user clicks to add particles to a mass-spring simulation; particles created near other particles are connected to them by springs that snap if stretched too far. In addition to this basic functionality, you will implement a few extra features such as a floor that stops particles from falling through, and saws that destroy all springs or particles they touch.

1.1 Starter Code

I have provided you with starter code, written in C++ and using the Polyscope, Libigl and Eigen libraries, as a starting point for implementing this project. You must implement the missing functionality in `main.cpp` according to this specification and the corresponding documentation in the code comments. Much of the UI and program skeleton has been written for you in the starter code. In principle you need only add code to the physics backend, and can leave the front-end entirely untouched.

1.2 Program GUI

The simulation will take place on a square piece $[-2, 2] \times [-1, 1]$ of the xy plane. Particles, springs, saws, the floor, etc. inside this square are rendered in a rectangular panel, and this is also where the user clicks to add new particles or saws.

Parameters of the program and simulation are controlled by a series of widgets, divided into several categories. The names in `fixed-width font` are those that will be used later in the spec to refer to the parameters, and these are also the names of the variables used for these parameters in the starter code.

1.2.1 Simulation Control

These control the behavior of the time integrator that advances virtual time according to the laws of physics.

- A button "Run/Pause Sim" toggles whether the simulation is paused, i.e. whether or not time advances in the simulation. While the simulation is paused, no physics takes place (but the user can still add particles/saws).
- A "Reset Sim" button deletes all simulation objects (particles, springs, saws) from the simulation, but leaves the GUI options unchanged.

1.2.2 UI Options

These options affect what happens when the user clicks on the world panel. In this milestone, there is only one option:

- A dropdown box (`clickMode`) toggles what happens when you left-click: either you add a particle or a saw.

1.2.3 Simulation Options

- The real "Time Step" parameter (`timeStep`) is the time step of the numerical integrator.
- A dropdown box (`integrator`) allows the user to select which time integrator to use: options are Explicit Euler, Implicit Euler, Implicit Midpoint, and Velocity Verlet.
- The real "Newton Tolerance" (`NewtonTolerance`) and integer "Newton Max Iters" (`NewtonMaxIters`) parameters control the number of iterations Newton's Method uses for the two implicit integrators.

1.2.4 Forces

The Forces panel controls which forces are active or ignored by the simulation, and allows the user to adjust various parameters of these forces.

- A series of checkboxes (`*enabled`) control whether or not Gravity, Springs, the Floor, and Viscous Damping is enabled in the simulation.
- For the gravity force, the real "Acceleration" parameter (`gravityG`) is the acceleration due to gravity felt by all non-fixed objects in the simulation.
- The real "Max Strain" (`maxSpringStrain`) parameter affects how much each spring can stretch before it snaps.
- Finally, the viscous damping has a real "Viscosity" parameter (`dampingStiffness`) controlling the rate of dissipation.

1.2.5 New Objects

This panel controls the creation of new simulation objects (particles, springs, and saws). Unlike the options in the above sections, which are continuously read by the simulation code, the parameters below are "locked in" each time the user creates a particle or spring.

- An "Is Fixed" checkbox (`particleFixed`), if checked, causes new particles to be anchored in place (unaffected by any forces).
- The real "Mass" parameter (`particleMass`) specifies the mass of any new particles.
- Whenever a new particle is created, springs are also created to any nearby particles. The real "Max Spring Dist" (`maxSpringDist`) parameter determines what is meant by "nearby."
- The "Base Stiffness" (`springStiffness`) is used to control how stiff the springs are in the simulation. (Each spring will have a slightly different stiffness, depending on its rest length; see the detailed spec below.)
- Lastly, the real "Radius" parameter (`sawRadius`) allows the user to specify the radius any newly-created saws.

2 Submission Instruction

All submissions must be uploaded to Canvas by the project due date and time. If you are working in a team, exactly one member of the team should upload the assignment. If Canvas is not working, or for some other reason you are unable to upload the assignment, you may turn in the assignment to the TA by emailing your project as a single .zip or .tar file. There will be *no exceptions* to the project due date–late submissions will be penalize per the syllabus late policy.

In addition to your complete source code, the compressed archive should include a `README` file containing, at minimum:

1. Your name and the name of your teammate (if any);
2. *Complete* instructions for how to compile your code, if you make changes to the build process;
3. Special instructions for how to run your code, if needed;
4. A description of anything you want considered for Creative Component extra credit, along with detailed instructions for how to access any extra functionality you've added;
5. An explanation of any required features that you know is not working completely correctly, but would like to have considered for partial credit;
6. Anything the spec below asks you to include.

Reminder About Teams Per the syllabus, you may work on this project in self-selected teams of up to two. If you work in a team, in addition to the code submission process described above, each team member must submit an individual Collaboration Report on Canvas (either as raw text in the submission system, or as an attached `.txt` or `.pdf`) with answers to the following questions:

1. In your estimation, what percent of the work did each team member contribute?
2. Did you pair-program, or did you split up the project into individual tasks? Which parts of the project did each team member work on? (3-4 sentences).

3 Grading

The milestone is broken down into several features described below. Each feature is worth a number of points and will be graded separately (although later feature may build on and require a working implementation of earlier features). Your code should be correct and free of segfaults, deadlocks, or other buggy behavior. You will *not* be graded on software engineering (use design patterns and unit tests if they help you; keep in mind that there will be a second milestone to this project, so well-written code may pay off down the line) but unreadable code is less likely to receive partial credit. You should not worry too much about performance or optimizing your code, but egregiously inefficient implementations of any of the features will be penalized.

4 Technical Component *(100 points)*

4.1 Basic Spring UI *(10 points)*

Implement the basic functionality of adding particles and springs to the simulation.

- *(2 pts)* When the user left-clicks on the world panel, if the `clickMode` is set to "Add Particle" rather than "Add Saw," add a particle to the simulation. This particle's initial position should be at the mouse cursor, it should have zero initial velocity, and its mass is specified by the current value of the `particleMass` parameter. (Different particles can have different masses, but a particle's mass, once set, does not change after creation.) Add a data structure to the simulation that keeps track of each particle's mass and current position and velocity, and render the particles so that there is an indication of the relative masses of the different particles. (This entire step has already been implemented for you in the starter code.)
- *(8 pts)* I have written a data structure that keeps track of the springs in the simulation, and does some basic spring rendering. Write code to populate springs when new particles are added to simulation. Each spring needs to know the two particles it connects, and its rest length (its stiffness can change dynamically according to the current value of the `springStiffness` parameter; see the section on forces below.) Whenever you add a particle, search the world for all other particles whose distance

to the new particle is at most `maxSpringDist`. Attach a spring between the new particle and all such particles. The rest length of a new spring connecting particle $\mathbf{p}_1$ to particle $\mathbf{p}_2$ has rest length $\|\mathbf{p}_2 - \mathbf{p}_1\|$. Once set, a spring's rest length does not change for as long as it remains in the simulation.

4.2 Configuration Vector Building *(5 points)*

Write code that takes the current positions of all of the particles and assembles one big *configuration vector* $\mathbf{q}$ out of those positions. Similarly, build a configurational velocity vector $\dot{\mathbf{q}}$ from all of the particle velocities. (Note that the number of particles in the simulation, and hence the dimensions of $\mathbf{q}$ and $\dot{\mathbf{q}}$, will change over time.)

Next, write code to do the opposite of the above: take in a configuration vector $\mathbf{q}$ and configurational velocity $\dot{\mathbf{q}}$, and store the values in the particle data structures for rendering.

4.3 Force and Force Derivative Computation *(20 points)*

For each of the two potentials below, write code which takes in the current value of the configuration $\mathbf{q}$ and computes the configurational force vector F^T (this will be a vector of the same dimension as $\mathbf{q}$). You may find it helpful to work out the forces in local coordinates first, and then map those to global (configurational) coordinates using selection matrices, as discussed in class. Also write code which takes in $\mathbf{q}$ and computes the force Jacobian (energy Hessian) $\left[dF^T(\mathbf{q})\right]$.

Again, it is probably helpful to work in local coordinates first. Notice that the (i, j) entry of the matrix $\left[dF^T\right]$ is nonzero only if the force F involves an interaction between $\mathbf{q}_i$ and $\mathbf{q}_j$; the matrix is therefore mostly zeros. To take advantage of this sparsity and save a lot of memory and computational time, **use sparse matrices** to encode $\left[dF^T\right]$. I recommend Eigen's `SparseMatrix<double>` class. You might find `SparseMatrix<double>::setFromTriplets()` particularly useful.

- *(5 pts)* Compute F^T and $\left[dF^T\right]$ for the gravity potential

$$V = \sum_{\text{particles } i} -m_i g y_i$$

  where m_i is the particle's mass and g is the current value of the `gravityG` parameter.

- *(15 pts)* Compute F^T and $\left[dF^T\right]$ for the spring potential

$$V = \sum_{\text{springs } s_{ij}} \frac{k_{ij}}{2} \left(\|\mathbf{p}_j - \mathbf{p}_i\| - L_{ij}\right)^2,$$

  where spring s_{ij} connects a particle at position $\mathbf{p}_i$ to one at position $\mathbf{p}_j$, L_{ij} is spring rest length, and k_{ij} is the spring stiffness. Springs with longer rest length are weaker: use

$$k_{ij} = \frac{k_b}{L_{ij}}$$

  for the spring stiffness, where k_b is the value of `springStiffness` at the time the spring was created, and L_{ij} is the spring rest length.

4.4 Mass Matrix *(2 points)*

All numerical integrators require knowing the configurational inverse mass matrix M^{-1} in order to perform the velocity update step. Write code to compute the *sparse* matrices M and M^{-1}.

4.5 Numerical Integrators *(30 points)*

You now have all of the pieces to write a few numerical integrators. In the starter code, the function `simulateOneStep` will be called continuously at regular intervals of "real-world" time as long as the simulation is unpaused. In this function, the simulated world should take a step forward in time of size `timeStep`. (If the simulation is in the paused state, time should stand still: no forces act on any objects in the world, and no animations play. The user interface is unaffected, i.e. you can add new particles and springs while time is stopped.) The Forces checkboxes control which forces are currently active in the simulation; only include forces in the time integration if they are active. The `integrator` dropdown box controls which of the algorithms below are used to perform time integration:

- *(5 pts)* Using your code to compute $\mathbf{q}$, $\dot{\mathbf{q}}$, F and M^{-1}, implement Explicit Euler integration

$$\begin{aligned}\mathbf{q}^{i+1} &= \mathbf{q}^i + h\dot{\mathbf{q}}^i \\ \dot{\mathbf{q}}^{i+1} &= \dot{\mathbf{q}}^i + hM^{-1}F(\mathbf{q}^i)^T\end{aligned}$$

  and store the updated positions and velocities back in the particle data structures.

- *(5 pts)* Implement Velocity Verlet integration

$$\begin{aligned}\mathbf{q}^{i+1} &= \mathbf{q}^i + h\dot{\mathbf{q}}^i \\ \dot{\mathbf{q}}^{i+1} &= \dot{\mathbf{q}}^i + hM^{-1}F(\mathbf{q}^{i+1})^T.\end{aligned}$$

- *(10 pts)* Implement Implicit Euler integration

$$\begin{aligned}\mathbf{q}^{i+1} &= \mathbf{q}^i + h\dot{\mathbf{q}}^{i+1} \\ \dot{\mathbf{q}}^{i+1} &= \dot{\mathbf{q}}^i + hM^{-1}F(\mathbf{q}^{i+1})^T.\end{aligned}$$

  You will need to use Newton's Method to solve this nonlinear pair of equations. Run Newton's method for `NewtonMaxIters` iterations, or until the residual $\|\mathbf{q}^{i+1} - \mathbf{q}^i - h\dot{\mathbf{q}}^i - h^2M^{-1}F(\mathbf{q}^{i+1})^T\|$ is less than `NewtonTolerance`, whichever comes first. Newton's method requires solving a linear system of the form $Ax = b$ for x. Do *not* solve this system by computing A^{-1} explicitly! Instead, use a numerical linear algebra algorithm to solve for x efficiently without computing A^{-1}–the Eigen library comes with several black-box sparse linear solvers. (*Debugging Hint*: Newton's Method should only require single-digit iterations for simple mass-spring networks. If you find that your code is taking a large number of iterations, that is an indication that you have a bug in either the force differential computation, or in the Newton's Method code. Use finite differences (described in the course notes) to test your derivatives for bugs.)

- *(10 pts)* Implement Implicit Midpoint integration

$$\begin{aligned}\mathbf{q}^{i+1} &= \mathbf{q}^i + h\frac{\dot{\mathbf{q}}^{i+1} + \dot{\mathbf{q}}^i}{2} \\ \dot{\mathbf{q}}^{i+1} &= \dot{\mathbf{q}}^i + hM^{-1}F\left(\frac{\mathbf{q}^{i+1} + \mathbf{q}^i}{2}\right)^T.\end{aligned}$$

  You will again need Newton's method, since these equations are implicit. Derive the right iterative algorithm for using Newton's method on these equations, and implement it, using the same tolerance parameters as for Implicit Euler.

4.6 Fixed DOFs *(2 points)*

By this point, you should have a working physical simulator for a nontrivial physical system. Congratulations! The remainder of the assignment adds on a few bells and whistles.

Implement "fixed" particles that do not move, and so can be used as anchor points off of which to build mass-spring networks: if the `particleFixed` check box is checked, any new particles created are fixed. These particles should have special rendering, and should be completely unaffected by any of the forces in the simulation.

4.7 Viscous Damping *(10 points)*

In the real world, springs that are set in motion do not keep oscillating forever: various non-conservative forces, such as drag against the air, and friction within the metal of the spring, damp out the motion of the spring over time. You will now implement a damping force for the springs in your simulation.

Viscous forces are those that oppose the *rate* at which objects in the simulation are moving (as opposed to the *elastic* forces you implemented above, which cared about the *amount* you had moved the objects apart, but not the rate at which you did so.) A simple approximation of the viscous force that acts to damp the oscillations in a spring connecting points $\mathbf{p}_1$ and $\mathbf{p}_2$ with velocities $\dot{\mathbf{p}}_1$ and $\dot{\mathbf{p}}_2$ is

$$F_1(\dot{\mathbf{p}}_1, \dot{\mathbf{p}}_2) = k_{\text{damp}}(\dot{\mathbf{p}}_2 - \dot{\mathbf{p}}_1)^T$$
$$F_2(\dot{\mathbf{p}}_1, \dot{\mathbf{p}}_2) = k_{\text{damp}}(\dot{\mathbf{p}}_1 - \dot{\mathbf{p}}_2)^T,$$

where F_1 is the force on $\mathbf{p}_1$, F_2 is the force on $\mathbf{p}_2$, and k_{damp} (specified in the simulation by the parameter `dampingStiffness`) is a constant (with units of pressure times time) that controls the amount of damping.

Note that the above force is *not* conservative, since it depends on velocities instead of just positions. It is possible to incorporate it as-is into any of the explicit or implicit integrators you have implemented; however, doing so will require substantial modification to the code. An alternative is to approximate F by a difference of positions:

$$F_1(\mathbf{q}_1^i, \mathbf{q}_2^i) = k_{\text{damp}} \left(\frac{\mathbf{q}_2^i - \mathbf{q}_2^{i-1}}{h} - \frac{\mathbf{q}_1^i - \mathbf{q}_1^{i-1}}{h} \right)^T$$
$$F_2(\mathbf{q}_1^i, \mathbf{q}_2^i) = k_{\text{damp}} \left(\frac{\mathbf{q}_1^i - \mathbf{q}_1^{i-1}}{h} - \frac{\mathbf{q}_2^i - \mathbf{q}_2^{i-1}}{h} \right)^T,$$

where $\mathbf{q}^i$ are the positions at the current time step, $\mathbf{q}^{i-1}$ are the positions at the *previous* time step, and h is the time step size. Note that this force now depends only on positions and not velocities; nevertheless, it cannot be conservative since it now includes *history-dependent* constants $\mathbf{q}^{i-1}$.

Implement this damping force (and include that force in the time integrator if `dampingEnabled` is true). With care it should be possible to implement this force in the same framework as you used for the other forces, i.e. by writing code that computes the configurational force, given values for $\mathbf{q}^i$ and $\mathbf{q}^{i-1}$, and the configurational force differential, given $\mathbf{q}^i$ and $\mathbf{q}^{i-1}$. For the Explicit Euler integrator, $\mathbf{q}^{i-1}$ is simply the position at the beginning of the previous time step. Add support to your code to keep track of this previous position (a newly-created particle's previous position is just its current position). You will need to work out the correct way of handing $\mathbf{q}^{i-1}$ in the other integrators.

4.8 Floor Force *(5 points)*

Without special handling, gravity will accelerate all loose particles off of the bottom of the screen. Add a new force, of your own design, to the simulation (and include that force in the time integrator if the force is enabled, i.e. if `floorEnabled` is true) that simulates a floor along the line $y = -0.5$. Render the floor, if the floor force is active.

You may choose any conservative or non-conservative force, provided that it meets the following requirements:

- It works with all time integrators. This means you will need to implement its differential, for the two implicit methods.

- It does nothing to particles whose y-coordinate is greater than -0.5. In other words, the force acts *only* on particles touching, or below, the floor.
- The force is strong enough to prevent particles, and clusters of particles, of total mass ≤ 100 kilograms from falling through the bottom of the screen ($y = -1.0$) when using default values for all other simulation parameters.

Document the force you chose, and a brief rationale, in the `README` file. Ideally the force will cause dropped particles to eventually come to rest close to the floor at $y = -0.5$, but you do not need to achieve this behavior at this time to get full credit. Particularly good solutions to the problem of resolving collisions against the floor will count towards the extra-credit Creative Component.

4.9 Spring Snapping *(5 points)*

If a spring becomes too stretched, it snaps: it is deleted from the simulation. The *strain* of an elastic object is a dimensionless quantity that measures how much the object has been stretched; for a one-dimensional spring, the (Cauchy) strain is

$$\epsilon = \frac{L - L_0}{L_0}$$

where L is the current length of the spring, and L_0 is the rest length.

After every time integration step, calculate ϵ for each spring. Delete from the simulation any springs whose current strain exceeds `maxSpringStrain`.

4.10 Add Saws *(10 points)*

When `clickMode` is set to "Add Saw" instead of "Add Particle" mode, left-clicking on the world panel should place a saw at the mouse position instead of a particle. In addition to a position, each saw has a radius, which is set to `sawRadius` at the time the saw is created and remains constant thereafter. Different saws in the simulation can have different radii.

For the purpose of the simulation, each saw is a disk (filled circle) of the given radius. The position of the saw is fixed (completely unaffected by the forces). Any particle whose center enters the disk is destroyed; if a spring connects two particles $\mathbf{p}_1$ and $\mathbf{p}_2$, and any part of the line segment containing $\mathbf{p}_1$ and $\mathbf{p}_2$ enters within the disk, the spring is destroyed.

I have implemented the UI for adding saws: left-clicking in "Add Saw" mode places a saw of radius `sawRadius` at the current mouse position, which remains in place until the simulation is reset. The starter code renders the saws so that the position and radius of each saw is obvious.

- *(2 pts)* Write a helper function which, given a point $\mathbf{p}$ and a pair of points $\mathbf{q}_1, \mathbf{q}_2$, calculates the shortest distance from $\mathbf{p}$ to the infinite line passing through $\mathbf{q}_1$ and $\mathbf{q}_2$. (*Hint*: if $\mathbf{c}$ is the closest point on the line to $\mathbf{p}$, then the vectors $\mathbf{c} - \mathbf{p}$ and $\mathbf{q}_2 - \mathbf{q}_1$ are perpendicular (do you see why?). Write $\mathbf{c} = \mathbf{q}_1 + t(\mathbf{q}_2 - \mathbf{q}_1)$ for some unknown scalar t, and derive the formula for t, then for $\mathbf{c}$, then for the minimum distance.)
- *(2 pt)* Write a helper function which, given $\mathbf{p}$ and $\mathbf{q}_1, \mathbf{q}_2$, calculates the shortest distance from $\mathbf{p}$ to the *finite* line segment connecting $\mathbf{q}_1$ to $\mathbf{q}_2$.
- *(2 pts)* Write a function that detects all springs and all particles that are being destroyed by at least one saw.
- *(4 pts)* After performing every time integration step, compute which springs and particles are touching the saws (using the above functions) and remove them from the simulation. (If a particle is removed, all springs connected to it are removed as well.)

4.11 Miscellaneous *(1 point)*

- Ensure that the "Reset Sim" button correctly and cleanly removes all particles, springs, and saws from the simulation.
- If a particle travels far from the visible area of the world, delete it as if it had been hit by a saw.

5 Creative Component *(up to 15 points extra credit)*

You may earn extra credit on this project (and possibly receive a project grade of over 100%) by going above and beyond the requirements of the Technical Component listed above, in whatever way you desire. The amount of extra credit you receive will depend on the following factors:

- How well the extra features complement and improve those already required;
- Aesthetics;
- Technical difficulty;
- Originality.

Some examples of improvements that would earn extra credit include drastically improving the rendering over that of the reference implementation (which might charitably be described as "minimalist"); gamification of the simulation by adding specific goals, limitations, levels, etc. to the basic program; researching and incorporating additional physics (friction; better collision detection and response; more sophisticated numerical integrators (Runge-Kutta 4; Lobatto IIIA-IIIB)), etc. I might show off anything submitted as a Creative Component during a future class, or incorporate it into future starter code.

Important: Describe in detail any features you added that you want considered for extra credit in the `README` file. If your creative component modifies in any way the required functionality described in the Technical Component, you must include (and describe in the `README`) a way to toggle between "vanilla" and enhanced functionality–this could be a menu option, a completely separate build, etc.

6 Reminder about Collaboration Policy and Academic Honesty

You are free to talk to other students about the algorithms and theory involved in this milestone, or to get help debugging, but all code you submit (except for the starter code, and the external libraries included in the starter code) must be your own. Please see the syllabus for the complete collaboration policy.